

Full Length Article

Using a single actor to output personalized policy for different intersections

Kailing Zhou, Chengwei Zhang^{id*}, Furui Zhan^{*}, Wanting Liu, Yihong Li

Dalian Maritime University, 1 Linghai Road, Dalian, 116026, Liaoning, China

ARTICLE INFO

Keywords:

Multi-agent reinforcement learning
Adaptive traffic signal control
Personalization policy
Execution efficiency

ABSTRACT

Recent advances in Multi-Agent Reinforcement Learning (MARL) have demonstrated significant potential for adaptive traffic signal control. However, existing MARL approaches face dual challenges: Complete parameter sharing among agents leads to insufficient diversity in policy networks, while treating agents as heterogeneous entities (with non-shared parameters) better adapts to intersection heterogeneity but introduces training inefficiency and parameter explosion in large-scale road networks. Balancing agent performance with reduced computational resource consumption remains a critical challenge that demands urgent resolution. To address these issues, we propose the Hyper-Action Multi-Head Proximal Policy Optimization (HAMH-PPO) method, which enhances personalized representation capabilities through value functions. This approach constructs K shared value function libraries to provide differentiated value estimation for traffic networks, while employing a hyper-network to dynamically generate adaptive weights for these libraries across different intersections. Consequently, a single shared network can learn diverse control strategies that accommodate varying observations. The experimental results showed that HAMH-PPO outperforms traditional traffic signal methods by 39.44% and achieves a 29.8% improvement over the suboptimal solution in complex, large-scale synthetic road networks. Crucially, HAMH-PPO maintains the benefits of parameter sharing while optimizing algorithmic performance and reducing computational costs.

1. Introduction

Multi-Agent Reinforcement Learning (MARL) focuses on collaboratively training a group of agents to solve specific tasks in shared environments. Recent research focuses primarily on optimizing feature extraction techniques (Ge et al., 2021; Qi et al., 2024), refinement of agent modeling (Wu et al., 2021), and design of hierarchical cooperative strategies (Abdoos & Bazzan, 2021; Shen et al., 2023; Xu et al., 2021). However, a crucial, yet often overlooked issue is how to effectively scale multi-agent algorithms for scenarios with a large number of agents, especially in complex Traffic Signal Control (TSC) applications. Statistics show that 96% of related studies involve simulations with fewer than 100 intersections (Liu et al., 2023b).

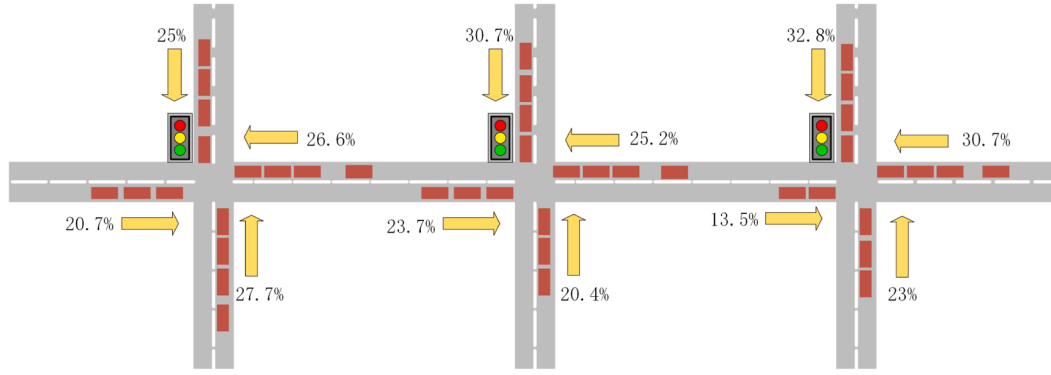
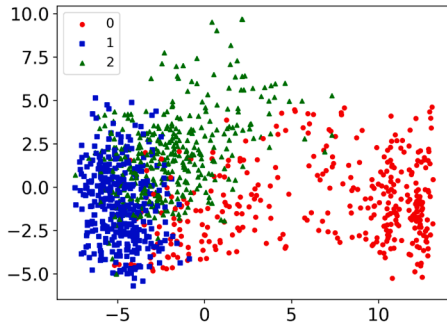
To train multi-agent systems for large-scale TSC, parameter sharing (Christianos et al., 2021), a common technique where network parameters are shared among agents, has proven highly effective, achieving remarkable results in road networks with more than 2000 intersections (Chen et al., 2020; Wei et al., 2019b). This effectiveness is mainly due to the similar environmental characteristics of many intersections,

which allow agents to find similar observations and reward functions. This similarity enables uniform representation of agents across neural network layers, speeding up training. Although parameter sharing accelerates training, it forces all intersections to adopt a unified value representation, overlooking their individual characteristics. This approach creates conflicting gradient signals when handling intersections with distinct traffic patterns, forcing the network to learn compromised value averages that fail to capture the individual characteristics of the intersections.

To address this issue, some methods (Chu et al., 2019; Liu et al., 2023a; Zhang et al., 2022a) aim to personalize agent behaviors by distinguishing between agents. However, this contradicts the goal of solving large-scale problems, as it is impractical to design unique parameters for each agent in extensive networks. There are also works (Liu et al., 2023b; Ma & Wu, 2020) that attempt to improve agent diversity by grouping intersections and sharing policies within these groups, or introduced selective sharing mechanisms (Zang et al., 2024) that specialize policy parameters through value decomposition. Although these approaches can balance diversity and efficiency to a certain extent, they

* Corresponding authors.

E-mail addresses: chenvy@dlnu.edu.cn (C. Zhang), izfree@dlnu.edu.cn (F. Zhan).

(a) 1×3 road network structure and traffic flow distribution.

(b) The distribution of observations for the three intersections, processed through Principal Component Analysis (PCA) for dimensionality reduction.

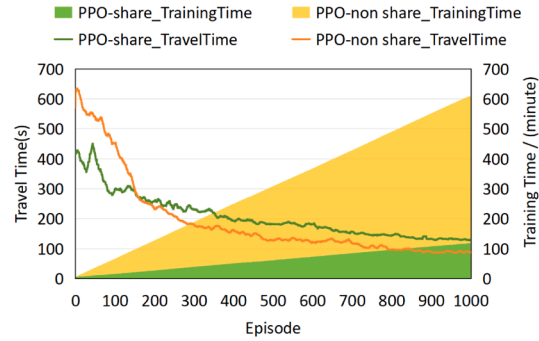
(c) Learning curves (measured by average travel time) and stacked chart of training duration (minutes) under parameter sharing (*PPO-share*) and non-parameter sharing (*PPO-non share*) methods.

Fig. 1. The traffic flow distribution and the observed distribution of a 1×3 network, as well as the learning curve for parameter sharing and non-parameter sharing methods (plotted based on 5 testing runs, with smoothing applied every 9 episodes.). This means that the personalization of the intersection needs to be considered.

depend on effective agent partitioning and overlook individual agent preferences. Thus, further research is needed to better leverage each agent's unique characteristics while maintaining system efficiency.

Here we show a simple demo (Fig. 1(a)) that simple parameter sharing becomes ineffective in complex and irregular traffic environments, indicating that forming a unified hidden representation in such environments is difficult and indiscriminately considering the state characteristics of all intersections is unreasonable. The demo features an adaptive traffic signal control (ATSC) task with three consecutive intersections on a straight road, each having distinct traffic flow distributions. The heterogeneity of the traffic flow distribution can also be understood as the heterogeneity of the observation distribution in reinforcement learning, which results in agents having different optimal policies. As shown in Fig. 1(a), the proportions of vehicles approaching each intersection from four directions vary. For example, the yellow arrow at the first intersection indicates that 20.7% of vehicles pass through this lane. Red rectangles denote vehicle entities.

To illustrate the heterogeneity of traffic flow distributions, this study collected observational data over 360 consecutive time steps within a training episode and used Principal Component Analysis (PCA) to reduce the 12-dimensional data from each intersection to a two-dimensional space. PCA projects the data into a new coordinate system via a linear transformation to maximize variance. The abscissa (PC1) represents the most significant direction of variation, while the ordinate (PC2) represents the second most important direction. As shown in Fig. 1(b), the two-dimensional space clearly reveals a distinct clustering of data points from three intersections, highlighting their distributional disparities. By extracting principal components with maximum variance, PCA retains

essential features while reducing high-dimensional data complexity and computational costs. To highlight the necessity of intersection diversity and personalized policies, we compared two experimental setups: *PPO-share* (training a single agent for parameter sharing) and *PPO-non share* (training an independent agent for each intersection). Both setups are based on the PPO algorithm (Schulman et al., 2017) and are trained over 1000 episodes. Fig. 1(c) shows the training results with average travel time. The results indicated that *PPO-share* has a training time of only 119 minutes, significantly less than the 492 minutes required by *PPO-non share*. Initially, *PPO-share* quickly finds a relatively good policy. However, as training progresses, its limitations become apparent as a result of its inability to fully account for the unique characteristics of each intersection, making it difficult to achieve optimal results. In contrast, *PPO-non share*, despite its slower start, ultimately yields better performance. These findings underscored the importance of considering intersection diversity and adopting individualized strategies in complex traffic networks. Balancing training efficiency with the individual requirements of intersections will be a key focus of our research.

Our goal is to develop a cooperative joint policy training framework and model mechanism for ATSC scenarios that enables efficient and robust policy learning through parameter sharing between multiple intersections while avoiding the pitfalls of traditional sharing methods. Local observations with varying distributions often lead agents to suboptimal local solutions, compromising overall performance across intersections, a phenomenon similar to *client drift* (Karimireddy et al., 2020).

To better address personalized requirements under shared parameters, we propose a multiagent reinforcement learning algorithm for TSC called Hyper-Action Multi-Head Proximal Policy Optimization

(HAMH-PPO). HAMH-PPO balances diversity and efficiency using the centralized training and decentralized execution (CTDE) paradigm, where agents share parameters. Unlike the standard CTDE setting, where the actor and critic output only one action distribution and value, HAMH-PPO's critic outputs multiple values for policy estimation, and its actor outputs an extra hyper-action—a probability distribution that weights the critic's output to provide personalized policies for different agents. Notably, hyper-action serves as a dynamic, instance-specific weight generator. Unlike prior approaches relying on static grouping or fixed masking, our hyper-action is generated in real-time by the network, with its weight distribution conditioned on each agent's unique real-time observations, historical states, and identity. This transforms policy personalization from being pre-defined and rigid to being adaptive, capable of adjusting to the varying traffic conditions at each intersection. To achieve this, HAMH-PPO's critic uses multi-head value functions with graph attention mechanisms to dynamically capture spatial dependencies between intersections and produce differentiated value function estimations. This ensures independent evaluation of each intersection, minimizing interference. To generate personalized hyper-actions, the actor in HAMH-PPO adds an additional one-hot code of the intersection index as input. This component is termed the hyper-action actor. This approach offers more comprehensive and accurate policy guidance, enhancing the model's representational capability. To avoid confusion with the actor network's 'action' (action applied to the environment), we refer to the hyper-network's output as a 'hyper-action'.

We evaluated the performance of the algorithm with both real and synthetic datasets, especially in a large-scale road network with 100 intersections. The experimental results showed that our proposed algorithm can flexibly adjust the weight according to the dynamic characteristics of the intersection, and the learned policy can effectively deal with the complex traffic environment. In summary, the main contributions of this work are as follows:

- We analyze and verify that there are differences in the observed distribution of intersections in TSC scenarios, and these differences have a direct impact on the performance of signal light control policies.
- A personalized MARL algorithm for ATSC, *i.e.*, HAMH-PPO, is proposed that achieves the balance between diversity and efficiency of TSC through the critic network of multivalued functions and the hyperaction mechanism. In addition, the model can provide personalized policy services for agents with different observation distributions.
- We conducted experiments on six datasets, including two real road networks and two synthetic road networks, to verify the efficiency and diversity of this method. We release our source code at <https://github.com/ET-zkl/HAMH-PPO.git>.

The remainder of this paper is organized as follows. Section 2 discusses related work. Section 3 describes the basic notations of ATSC and MARL. The proposed algorithm is presented in Section 4, and the experiments and performance analysis are detailed in Section 5. Finally, we conclude the article in Section 6.

2. Related work

Traditional TSC methods, mainly based on expert rules, cannot adaptively adjust signal phases according to real-time traffic flow. In recent years, MARL-based ATSC algorithms have shown promise. IntelliLight (Wei et al., 2018) emphasizes the importance of features, with observations including lane queue lengths, total vehicles at intersections, updated waiting times, vehicle position images, current and next actions. Colight (Wei et al., 2019b) first uses graph attention networks (GAT) (Velickovic et al., 2017) to learn the dynamic impact of neighboring intersections and capture spatial characteristics. PDA-TSC (Fang et al., 2022) addresses fairness issues and uses LSTM to predict the duration of action. MARL-DSTAN (Huang et al., 2021) and STMARL (Wang

et al., 2020) use graph attention and convolution to capture spatial features, while RNN extracts temporal dependencies. GCQN-TSC (Yan et al., 2023) is a graph cooperation Q-learning method designed for ecological traffic. Taking into account the heterogeneity of intersections, IG-RL (Devailly et al., 2021) uses GCNs (Hamilton et al., 2017) to obtain parameters suitable for various road network controls, enhancing the generalization and transferability of the agent. IHG-MA (Yang et al., 2021) generates embeddings for new vehicles and traffic networks, encoding both heterogeneous node characteristics and structural information. CoevoMARL (Chen et al., 2024) uses RDP-LSTM to dynamically evolve the spatial interaction network of signals. CCGN (Mukhtar et al., 2023) combines local and global policy networks for signal-free traffic flow control. CI-MA (Yang et al., 2023) uses causal inference modeling to handle non-stationarity, resulting in effective cooperative strategies.

In these methods, multiple agents share a learning network, significantly increasing the efficiency of neural network training. However, they overlook the individuality of intersections, potentially degrading performance. For example, MA2C (Chu et al., 2019) trains independent agents for each intersection, controlling traffic signals through local observation and neighbor communication. PressLight (Wei et al., 2019a) optimizes rewards, but struggles with global optimization as it cannot holistically consider all relevant intersections. PNC-HDQN (Zhang et al., 2022a) decouples intersection-related calculations from RL agent training, enabling RL algorithms to handle dynamic data changes. Zhang et al. (2021) proposed a neighborhood cooperative Markov game framework. It assigns each intersection the objective of maximizing its neighborhood's average cumulative payoff and employs a forgetful experience mechanism to diminish past experience importance. However, fully considering agent individualization in large-scale traffic scenarios is impractical due to inefficient model training. The leader-follower paradigm (Zhang et al., 2022b) is also an effective method to distinguish between heterogeneous agents. Zhang et al. (2022b) applied this model to ATSC scenarios with two types of agents controlling intersections. However, given the often distinct traffic flow distributions at each intersection, the role distinction may not capture specific intersection characteristics. RegionLight (Gu et al., 2024) achieves regional intersection coordination by structuring the intersection topology into a star shape (one central node connected to multiple leaf nodes). However, the requirement of this method for manual regional division limits its flexibility in large-scale road network scenarios.

In summary, ATSC is inherently a multi-agent problem, with each intersection acting as an independent agent. In these complex systems, the coordination mechanism between agents is crucial. It must account for the dynamic influence of neighboring intersections and accurately capture the characteristics of traffic flow of different periods within the same intersection. Real-world transportation networks often cover a large number of intersections that cannot be treated uniformly or in isolation. Therefore, it is necessary to explore a MARL method that can meet the diversity policy requirements and train efficiently.

3. Preliminary

3.1. Partially observable Markov decision process

In this section, the TSC problem is described as a partially observable Markov decision process using the basic problem defined in multi-intersection TSC (Wei et al., 2019b). Each intersection in the system is controlled by an RL agent, who observes a portion of the entire system. The goal of each agent is to adaptively control the phase change of the signal lights to minimize the cumulative number of vehicles waiting at the intersection where it is located for a long time. Define the problem as a tuple: $\langle S, O, A, P, R, \gamma \rangle$.

- System state space S and observation space O . Suppose that there are N intersections in the system and S is the state space used to describe all possible states of the system. $\{O_1, \dots, O_{|N|}\}$ is the

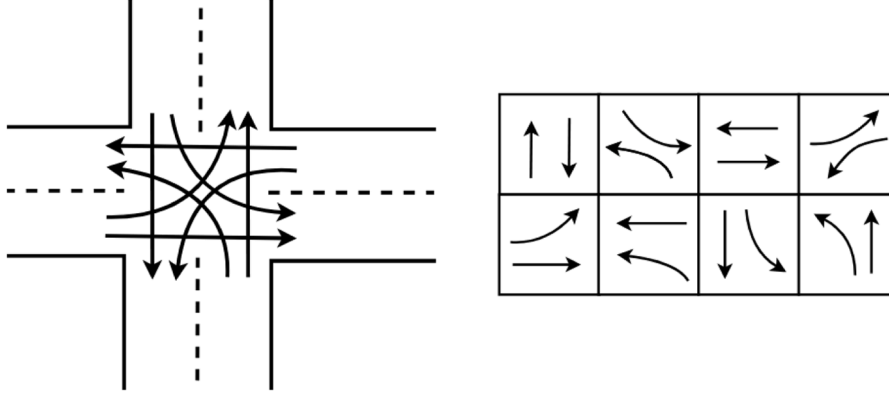


Fig. 2. Signal phase and corresponding action set of crossroads.

observation space and O_i of agent i is partially observed from the state of the system. The observation space should reflect the characteristics of the road network as accurately as possible and in line with real-world constraints. In a TSC scenario, the number of vehicles waiting in the incoming lanes can be an indication of the traffic situation in time. Therefore, the observation is defined as: $o_i^t = \text{wave}[l]_{l \in L_i}^t$, where L_i is the set of incoming lanes at intersections i . $\text{wave}[l]^t$ is the number of vehicles waiting in the incoming lane l at time t .

- Action space A . $\{A_1, \dots, A_{|N|}\}$ is the joint action space of all agents. At each time t , the agent i selects an action $a_i^t \in A_i$ as a decision for the next period. In this work, we follow the standard definition of action space, which is defined as a set of phases, i.e., a set of non-conflicting traffic movements. A traffic movement is a pair of one incoming lane and one outgoing lane. At the intersection shown in Fig. 2, there are eight phases in total.
- Transition probability P . The transition probability $P(s^{t+1}|s^t, \mathbf{a}^t) : S \times A_1 \times \dots \times A_N \times S \rightarrow [0, 1]$ is the transition probability space that assigns a probability to each transition.
- Reward R . The reward R_i for each agent i is obtained by the reward function $R_i : S \times A_1 \times \dots \times A_N \rightarrow \mathbb{R}$. The optimization goal in ATSC is to reduce the travel time of vehicles, which is difficult to quantify directly with the reward for each intersection in each step. Therefore, we used the sum of the waiting queue lengths of the vehicles in all lanes of the intersection as an alternative objective. In detail, the reward of the agent i at time t is formalized by $r_i^t = -\sum_{l \in L_i} \text{wave}[l]^t$.
- Policy π and discount factor γ . At time t , each agent chooses an action following a certain policy π . The aim is to learn an optimal joint policy $\pi^* = \pi_1^* \times \dots \times \pi_N^*$ to maximize the cumulative discount average reward $R^t = E[\frac{1}{|N|} \sum_{i \in N} \sum_{t=\tau}^T \gamma^{t-\tau} r_i^t]$, and $\gamma \in [0, 1)$ distinguish the rewards according to temporal proximity.

It should be noted here that in the model defined above, each agent has its own reward, which is different from the common multi-agent cooperation problem, where all agents share one reward. Obviously, the POMDP defined in this article is a mixed-game problem, in which multiple agents with different individual rewards cooperate with each other to maximize their average return. There are two reasons why we define TSC issues in this way. Firstly, explicitly designing rewards for each agent helps improve the learning efficiency of the algorithm, as it does not need to face the credit assignment problem in cooperation MARL scenarios. In addition, we have also demonstrated the rationality of this design in our previous works (Zhang et al., 2021, 2022a). Previous works show that there is a weak cooperative relationship between multiple intersections in the same road network, and their individual returns are positively correlated. Therefore, optimizing individual returns will also optimize the overall average return.

This environmental setting directly leads to the personalized needs of different agents. Because different agents in the same system state have different rewards, the value function estimates for each agent under the same joint policy should also be different, which further leads to personalized requirements for individual policies. As described above, we want all agents to share the same network to improve learning efficiency. The next question is how to design a MARL algorithm so that multiple agents with shared policy network can efficiently learn the optimal joint policy, while also ensuring the personalized needs of all agents.

3.2. PPO

PPO (Proximal Policy Optimization) (Schulman et al., 2017) is a popular reinforcement learning algorithm that combines the advantages of policy optimization and value estimation methods. Objectives to effectively optimize policies while ensuring stability and convergence by limiting the difference between the new policy and the old policy in each update, thus preventing the policy from deviating too far during training. Specifically, PPO introduces a clipping mechanism to limit the updating steps of policy and optimizes the policy parameter θ by maximizing the PPO-clip objective of

$$L^{\text{CLIP}}(\theta) = E[\min(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (1)$$

where θ is the policy parameter, E denotes the empirical expectation over timesteps. ρ_t is the importance sampling ratio, i.e., the probability ratio under the new and old policies formalized by $\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. A_t is the estimated advantage at time t and ϵ is a hyperparameter. The function $\text{clip}(a, b, c)$ clips the value a to the interval $[b, c]$. In this article, we propose a CTDE MARL algorithm based on the PPO framework, where all agents share parameters but have different preferences.

4. Hyper-action multi-head proximal policy optimization

In this section, we provide the implementation details of the Hyper-Action Multi-Head Proximal Policy Optimization (HAMH-PPO) algorithm. Fig. 3(a) shows the overall framework of HAMH-PPO. HAMH-PPO consists of a shared decentralized actor Fig. 3(b) and a shared centrally trained critic Fig. 3(c), and employs the CTDE paradigm to train agents to learn coordinated policies. Unlike the classical cooperative CTDE algorithm MAPPO, our algorithm provides estimates of the value function for each intersection, and the centralized training mainly involves training with global data combined together, because we need to estimate different state values for agents with different individual returns.

Specifically, in a TSC scenario with $|N|$ agents with a shared network parameter (a shared actor network and a shared critic network), where all agents make decisions based on their local observation history by the actor network and estimate their state values by a centralized critic

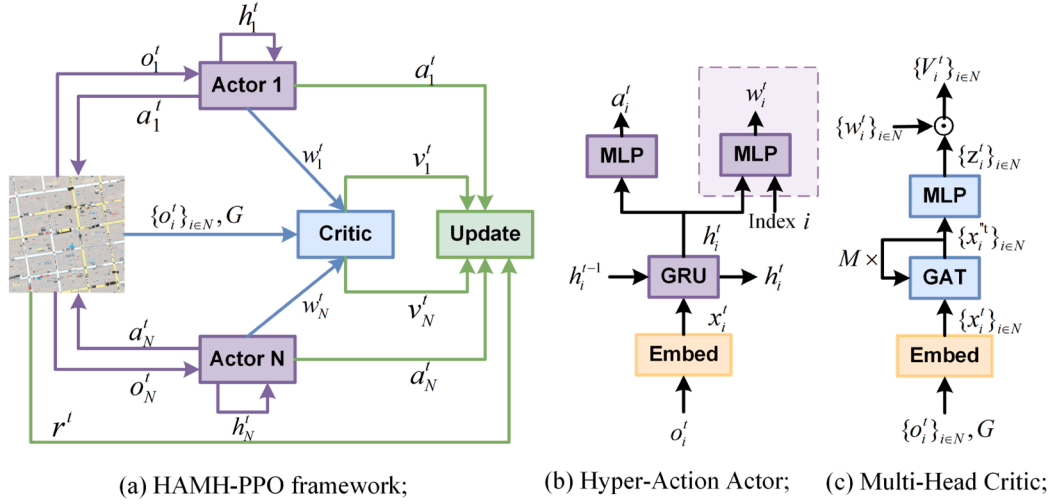


Fig. 3. The overall structure of HAMH-PPO is illustrated in Figure (a) and is designed based on the PPO algorithm. It consists of N agents, each corresponding to an intersection, and a centralized critic. Figure (b) shows the structure of a single actor, where local observations are passed through feature extraction and output action a_i^t that affects the environment, while cyclic features combined with the intersection index i output the hyper-action w_i^t . Figure (c) represents the structure of the critic, where global observations and graph information are processed using GAT to generate a set of value functions for each intersection. The joint value function, obtained by dot product with the hyper-action, estimates the average travel time at the current intersection.

network. The centralized critic network, shown in Fig. 3(c), is shared by all agents, estimating returns for each agent by accessing global information. Taking into account the obvious characteristics of the graph structure between intersections in TSC scenarios, the critic uses a graph attention neural network to calculate the graph representations of all intersections (nodes on the graph) in the road network. This graph representation is further processed through a shared MLP layer to output the state value of each intersection (agent). In addition, as shown in Fig. 1(b), it is noted that there are differences in the distribution of observation and state transitions of different intersections in the road network, and the sampling frequency of the corresponding data for the observation representation of different agents is different, resulting in different returns. Even if graph neural networks can produce different observation representations for different agents from one global system state, it is insufficient to use a single linear mapping (i.e., MLP layer) to estimate the values of all agents. To do this, the critic network in HAMH-PPO outputs a set of value functions z_i^t with set size $|z_i^t| = k$ for each agent which is implemented by an MLP layer with multiple heads. The network of decentralized execution actors, shown in Fig. 3(b), is also shared by all agents. They take the local observation history as input and extract the time characteristics of the intersection through GRU. The outputs are the distributions of action a_i^t and hyper-action w_i^t of the agent i . The action distribution is the same as the standard RL definition, which outputs the probability of selecting a certain action a_i^t in the action space, and the hyper-action w_i^t is also a probability used to balance the multiple values estimated from the centralized critic using a dot-product operation and further guide the updating of action policies. So we call the actor and the critic networks hyper-action actor (HA-ACTOR) and multi-head critic (MH-Critic), respectively. The following are the specific details of the two networks.

4.1. Hyper-action actor network (HA-ACTOR)

In the TSC problem, historical observations are critical. Therefore, HA-ACTOR uses the GRU to handle long sequential data to capture the temporal dependency at intersections. As can be seen in Fig. 3(b), for each agent i , the input of the actor network is the observation o_i^t and the historical representation h_i^{t-1} . A multi-layer perception is used to obtain the observation embedding x_i^t , followed by a GRU unit to capture

a temporal representation of the current state of agent i , formally,

$$x_i^t = \text{Embed}(o_i^t) = \text{ReLU}[W_c \cdot \text{ReLU}(W_d \cdot o_i^t + b_d) + b_c] \quad (2)$$

$$h_i^t = \text{GRU}(x_i^t, h_i^{t-1}) \quad (3)$$

where W_c , W_d , b_c and b_d are the weight matrix and bias vectors to learn. The hidden feature h_i^t outputs the action probability distribution through the linear layer on the left and then selects an action a_i^t based on probability sampling.

In HAMH-PPO, the core idea of personalization is personalized learning that generates distinct value estimation guidance policies for intersections. Compared to a single value function, a set of value functions can select different estimates for different intersections. However, selecting an estimation value function from a finite set of value functions for the current action may not be sufficient. One idea is to consider whether we can permit several value functions to jointly guide the update of the policy network. Thus, a hyper-action network is introduced, which takes the hidden features h_i^t and the intersection index i as input to generate weights w_i^t for each value function. We can provide numerous combinations of value functions, thus freeing the solution space from constraints. It should be noted that w_i^t is a probability distribution of the value functions, rather than a multidimensional action. Critically, unlike static grouping methods, this weight is dynamically conditioned on each agent's real-time state, which fundamentally shifts personalization from a fixed assignment to a dynamic and adaptive process. For convenience, we will designate the dashed portion in Fig. 3 as hp_φ .

$$w_i^t(\cdot | o_i^t, h_i^{t-1}, i) = \text{softmax}((h_i^t \oplus i)W_e + b_e) \quad (4)$$

where W_e and b_e are the weight matrix and bias vector to learn, h_i^t is the historical representation at time t output of the GRU unit. $\oplus$ is the concat operator.

The algorithm adopts a centralized training method to train actors. Due to parameter sharing, we can integrate the local observation data of all agents to jointly train the actor network. Given a trajectory of interactive data $\{ \langle o_i^t, a_i^t, o_i^{t+1}, r_i^t \rangle \}_{i \in N}^{t < T}$, the hyper-actor trains its network parameters θ based on the following averaged PPO (Schulman et al., 2017) loss,

$$L^{PPO}(\theta) = \frac{1}{T|N|} \sum_{t < T} \sum_{i \in N} \min(\rho_i^t(\theta)A_i^t, \text{clip}(\rho_i^t(\theta), 1 - \epsilon, 1 + \epsilon)A_i^t) \quad (5)$$

where A_i^t is the advantage value calculated from the Generalized Advantage Estimation (GAE) (Schulman et al., 2016). ρ_i^t is the

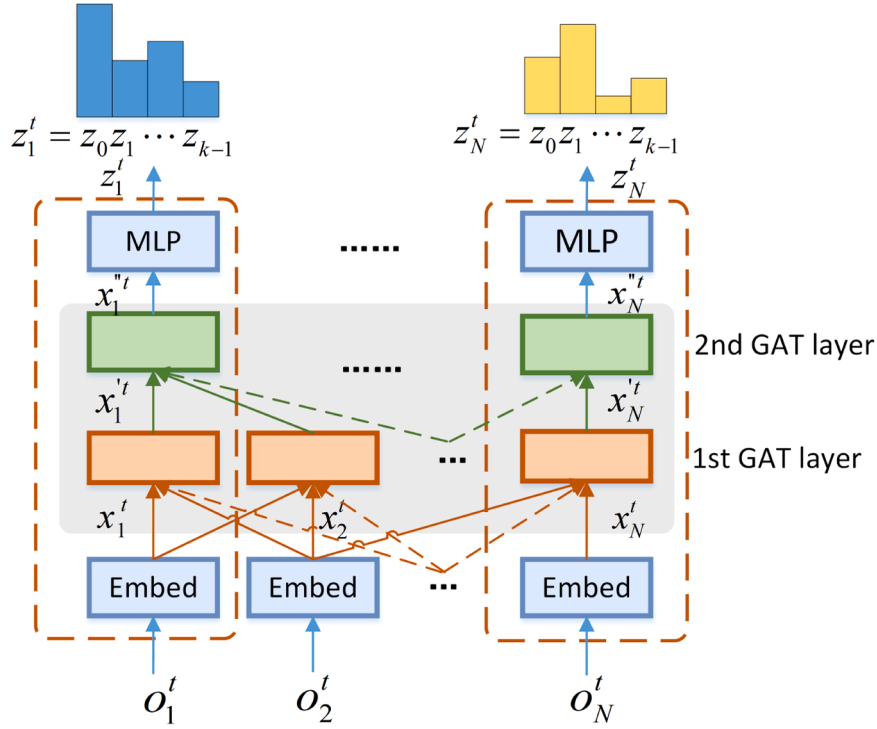


Fig. 4. The detailed structure of the cubic network, which ultimately outputs k -dimensional value estimates.

importance sampling ratio. In addition, to prevent the slow update of certain value functions due to excessively small individual weights, we added an entropy regularization term for the hyper-action distribution, i.e., $H(w_i^t(\theta))$, where $w_i^t(\theta)$ denotes the $w_i^t(\cdot|o_i^t, h_i^{t-1}, i)$ parameterized by θ . In conclusion, the current loss function of the actor network is:

$$L(\theta) = L^{PO}(\theta) - \lambda H(w_i^t(\theta)) \quad (6)$$

where λ is the weight entropy coefficient. The use of entropy regularization term in the new loss can enable the distribution of hyper-action to learn in the direction of increasing entropy, preventing it from tending towards a deterministic distribution prematurely.

4.2. Multi-head critic network (MH-Critic)

The centralized critic needs to access global information, which comprises the local observations of all agents and the adjacency relationships among graph nodes. We employ a Graph Attention Network (GAT) to process the graph structure and extract spatial features between intersections. Fig. 4 illustrates the detailed structure of the critic network, which includes agents N . The local observations of each agent are processed through a multilayer perceptron to obtain an observation embedding. To improve the overall system's generalization and performance, the actor and critic share an observation embedding layer, which not only reduces the network's parameter count, but also promotes better information sharing and complementarity between them. The observations of all agents are fed through an embedding layer to obtain the embedding features $\{x_i^t\}_{i \in N}$ of each intersection.

As shown in Fig. 4, the embedding features pass through a layer of GAT to combine neighbor information according to their importance and obtain $x_i'^t$. To increase the perceptual range of agents, we employ multiple GAT layers. In addition, more attention heads focus on different feature subspaces to capture richer relational representations, making training more stable. The extracted spatial features undergo dimensionality reduction and aggregation across multiple perceptions, ultimately yielding a vector z_i^t . Each intersection has its value function vector, representing multiple reward estimates for the current state. The set of

value functions serves as experts to guide the training of agents' policies, and our core idea is to have these experts learn in parallel and cooperatively to inform the policy network updates. Here, the hyper-actions $w_i^t = w_0 \dots w_{k-1}$ represent the probability distribution of data items in z_i^t . Our experiments show that as differences in traffic conditions at intersections increase, the disparities between these weights also increase. As a result, the proportions of value functions in policy updates vary, enabling personalized policy learning. Multi-valued functions and hyper-action share the same dimensionality k , and their dot product yields the final joint value function, providing precise and comprehensive guidance for optimizing the policy network. Formally, the value function of agent i at time t can be calculated as follows.

$$V_i^t = z_i^t(\cdot|\{o_i^t\}_{i \in N}, G) \odot w_i^t(\cdot|o_i^t, h_i^{t-1}, i) \quad (7)$$

where $\odot$ is the dot product operator of two vectors. z_i^t and w_i^t are the value distribution and hyper-action of agent i respectively. G is the connectivity matrix between the intersections used for GAT, which is fixed as a constant matrix in each traffic scenario in this article.

The loss function of the critic is consistent with PPO (Schulman et al., 2017), i.e., the TD squared error. Using a network φ to parameterize V , the loss function of the MH-Critic can be defined by the following.

$$L(\varphi) = \frac{1}{2NT} \sum_{t < T} \sum_{i \in N} [r_i^t + \gamma V_i^{t+1}(\varphi) - V_i^t(\varphi)]^2. \quad (8)$$

Algorithm 1 shows details of HAMH-PPO. HAMH-PPO trains two neural networks that all agents share: an actor network θ and a critic network φ . For each episode, the agents interact with the environment by executing their joint policy and collecting a set of trajectories (Step 4). Trajectories are stored in a replay memory D (Step 5), and all networks are trained according to Eq. 6 (Step 8) and Eq. 8 (Step 9). To increase data utilization and improve sample efficiency, we train E epochs on the same trajectory.

5. Experiments

In this section, we conduct extensive experiments to answer the following questions:

Algorithm 1 HAMH-PPO.

Input: Episodes K , Number of agents N , Time steps per episode T , Epoch E .

Output: Trained critic φ and actor θ .

```

1: Initialize critic-network  $\varphi$ , actor network  $\theta$ , Replay buffer  $D$ .
2: for  $k = 0, 1, \dots, K - 1$  do
3:   Reset simulator environment.
4:   Set data buffer  $D = \phi$ 
5:   Collect a set of trajectories by running the policy  $\pi_\theta$ .
6:   Store  $(o_i^t, a_i^t, r_i^t, o_i^{t+1})_{i \in N, t \in \{0, \dots, T-1\}}$  in buffer  $D$ 
7:   for  $e = 0, 1, \dots, E - 1$  do
8:     For each trajectory from  $D$ .
9:     Update  $\theta$  by Eq. 6.
10:    Update  $\varphi$  by Eq. 8.
11:   end for
12: end for

```

Table 1
Scenarios statistics.

Scenarios	Intersections	Arrival num (vehicles/300s)			
		Mean	Std	Max	Min
$Grid_{10 \times 10}$	100	880	0	880	880
$Grid_{4 \times 4}$	16	935.92	17.47	960	896
D_{Jinan1}	12	645.42	3.45	652	639
D_{Jinan2}	12	645.75	4.75	654	639
D_{Jinan3}	12	419.67	98.53	672	256
$D_{NewYork}$	48	235.33	5.59	244	224

- RQ1: How does our proposed method perform compared to other advanced methods?
- RQ2: Can HAMH-PPO generate personalized value estimates in road networks?
- RQ3: How do the components used in HAMH-PPO affect the performance of the algorithm?

5.1. Settings

We conducted experiments on the CityFlow traffic simulator. CityFlow is an open-source simulator widely used for TSC at multiple intersections. After entering traffic data in the simulator, the vehicle moves toward its destination according to environmental settings. The simulator provides status to the signal control method and executes traffic signal actions from the control method. Typically, there is a three-second yellow light signal and a two-second full red light signal after each green light signal. In our experiments, the simulation interval is set to 1.0 seconds, with each episode simulating 3600 seconds (one hour) of traffic. The speed limit on all roads is set to 40 km/h (approximately 11.11 m/s). To ensure statistical reliability, we repeat each experiment with five different random seeds.

5.2. Dataset

We validated the algorithm in 6 scenarios (Jinan $\times$ 3, New York $\times$ 1, synthetic road networks $\times$ 2) in two real and two synthetic road networks. Table 1 shows the distribution of traffic flow in each scenario. In all scenarios, each car has its own parameters, such as acceleration and maximum speed. Each road has three lanes: one left-turn lane and two through lanes. We set the phase number to 8 and the minimum action duration to 10 seconds.

Synthetic Data. In the synthetic dataset, we will use two types of maps. They are made up of different numbers of intersections. Each road at the intersection has three lanes with 3 m in width.

- $Grid_{10 \times 10}$. The road network has 100 (10×10) intersections, each of which is four-way. The structure of the road network is irregular. The east $\rightarrow$ west traffic flow is 900 vehicles/lane/hour and 150

vehicles/lane/hour, and the north $\rightarrow$ south traffic flow is 720 vehicles/lane/hour and 90 vehicles/lane/hour.

- $Grid_{4 \times 4}$. The road network has 16 (4×4) intersections. Each intersection is four-way, with two 300-meter (East-West) road segments and two 300-meter (South-North) road segments. The traffic flow is not regular.

Real-world Data. We also use real-world traffic data from two cities: Jinan and New York. The road network structures are imported from OpenStreetMap. Jinan includes three traffic-flow distributions with different arrival rates.

- D_{Jinan} . The road network has 12 (3×4) intersections. Each intersection is four-way, with two 400-meter (East-West) road segments and two 800-meter (South-North) road segments. In our experiment, this road network has three dataset settings.
- $D_{NewYork}$. The road network has 48 (3×16) intersections. Each intersection is four-way, with two 350-meter (East-West) road segments and two 100-meter (South-North) road segments.

5.3. Baseline

Our experiment mainly compared two types of methods: traditional TSC methods and RL-based signal control methods. For a fair comparison, all RL methods select the phase from the set of actions. Table 2 shows the parameter settings used in each RL algorithm. The details are as follows:

- **Fixedtime**(Koonce et al., 2008). Traffic signals at intersections operate according to a predetermined timing scheme, with traffic signals changing periodically.
- **MaxPressure**(Varaiya, 2013). The objective is to minimize the phase "pressure" at intersections by balancing the queue lengths between adjacent intersections, to minimize the phase pressure at each intersection, to maximize the throughput of the entire road network.
- **PNC-HDQN**(Zhang et al., 2022a). A fully decentralized neighborhood learning framework, where each agent preprocesses neighborhood data based on the correlation between two intersections.
- **MA2C**(Chu et al., 2019). A fully decentralized and scalable MARL algorithm where each local agent is optimized through coordination between multiple intersections.
- **MPLight**(Chen et al., 2020). A large-scale traffic road network algorithm using FRAP (Zheng et al., 2019) as a base model to train parameter-sharing agents using DQN.
- **Colight**(Wei et al., 2019b). By introducing the graph attention network, the dynamic influence of surrounding intersections on the current intersection is considered.
- **RegionLight**(Gu et al., 2024). An adaptive branch Duel q network (ABDQ) model is proposed to decompose the area control task into multiple joint signal control subtasks, each of which corresponds to a specific intersection.

5.4. Evaluation metric

The goal of TSC is to move vehicles through intersections more quickly, and the average travel time is the average time spent by all vehicles entering and exiting the road network. Therefore, we use the average travel time to evaluate the performance of signal control algorithms. This is also the most commonly used performance metric in the transportation domain. The average travel time of all vehicles:

$$m_H = \frac{1}{|v_{in}|} \sum_{v \in v_{in}} (t_v^{out} - t_v^{in}),$$

where t_v^{out} and t_v^{in} represent the entry and exit times of vehicle v , respectively, while v_{in} denotes vehicles entering the road network.

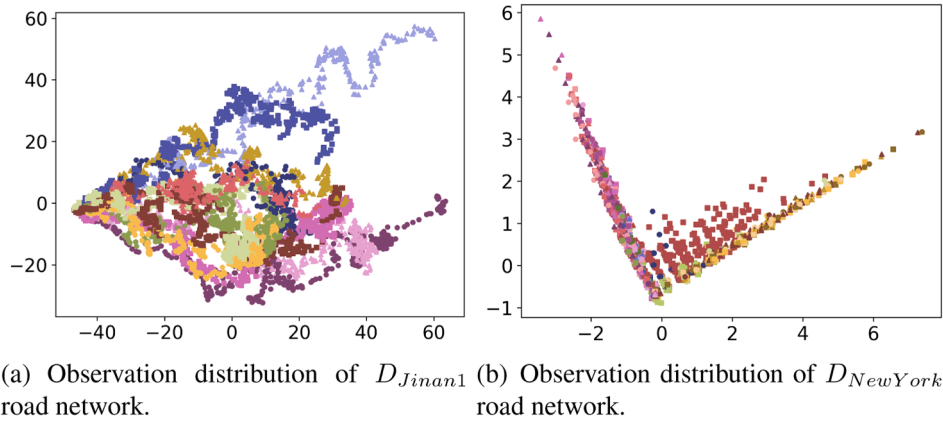


Fig. 5. Principal Component Analysis (PCA) is used to reduce the dimension of the observed intersection distribution of the two road networks. The observations were collected during the tenth training episode.

Table 2
Main hyperparameters of HAMH-PPO.

Parameter	Value
Hidden state dimension of GRU	128
Hidden state dimension of GAT	128
Layer numbers of GAT	5
Heads numbers of GAT	1
Actor learning rate α_θ	5e-4
Critic learning rate α_ϕ	5e-4
PPO clip ϵ	0.2
PPO epoch E	15
Discount factor γ	0.98
Network optimizer	Adam
Entropy coefficient λ	0.01
Dimension k of hyper-action	32

Table 3
Performance on synthetic data and real-world data.

Methods	$Grid_{4 \times 4}$	$Grid_{10 \times 10}$	D_{Jinan1}	D_{Jinan2}	D_{Jinan3}	$D_{NewYork}$
FixedTime	725.11	1440.88	537.64	537.76	514.14	1122.95
MaxPressure	431.76	953.98	798.89	694.90	374.56	203.08
PNC-HDQN	512.64	1170.48	727.73	584.65	374.78	1187.89
MA2C	677.12	1199.02	970.6	825.25	445.91	1215.44
MPLight	1047.59	1405.92	480.02	430.84	348.21	194.23
Colight	389.02	1100.49	635.55	502.32	335.89	186.49
RegionLight	338.70	1128.47	449.60	396.06	325.12	818.13
HAMH-PPO	273.90	669.34	427.44	366.26	306.45	181.69

5.5. Performance comparison (RQ1)

In this section, we will evaluate the performance of HAMH-PPO by comparing it with traditional transportation methods and RL methods across six scenarios. Fig. 2 summarizes the hyperparameter settings in HAMH-PPO.

Table 3 compares HAMH-PPO and seven other methods, including two traditional TSC methods and five advanced TSC methods for MARL. Compared to the two traditional methods, HAMH-PPO showed an average improvement of 48.21% over Fixedtime and 30.68% over MaxPressure in six datasets. This is because traditional TSC is not suitable for changing traffic conditions over time.

We also compared HAMH-PPO with five state-of-the-art MARL-based TSC methods. As shown in the table, our algorithm achieves the best results in all scenarios. Firstly, the traffic flow in the synthetic road network 4×4 is very large, and MPLight cannot handle this situation without the knowledge of the neighbors. The performance of HAMH-

PPO is 19.1% higher than that of RegionLight on this road network. Secondly, PNC-HDQN and MA2C are ineffective in the NewYork road network. These two algorithms train one agent for each intersection, which cannot deal with such large-scale, complex real road networks. The relatively small performance improvement of HAMH-PPO over MPLight and Colight in the NewYork road network can be attributed to the limited diversity in intersection observations. As shown in Fig. 5, the observation distribution of the 48 intersections in this network is highly consistent, indicating a lack of intersection diversity. This homogeneity in the observation distribution restricts the potential for significant performance gains from HAMH-PPO. Furthermore, the advantage of HAMH-PPO is more pronounced in the large and irregular synthetic 10×10 road network than in other maps. It achieves a 29.8% reduction in maximum travel time compared to the second-best scheme, highlighting its effectiveness in large-scale TSC scenarios. This is mainly because HAMH-PPO trains personalized strategies and selects appropriate value estimates for different intersections, thereby enhancing the model's representational ability. The experimental results showed significant improvements in vehicle traffic efficiency. In summary, HAMH-PPO can effectively adjust the hyper-action based on current road conditions and generate joint value estimates for each intersection, achieving excellent performance across various road networks, especially in large-scale TSC scenarios.

Fig. 6 presents the learning curves for PNC-HDQN, MA2C, MPLight, Colight, RegionLight and HAMH-PPO. The lines and shaded areas around the curves represent the average travel time and error range of these algorithms during the learning episodes. MA2C and PNC-HDQN, which train an agent for each intersection, exhibit a similar convergence behavior, taking a relatively long time to converge, with 200 episodes still insufficient for convergence. Similarly, RegionLight, which trains regional intelligent agents and coordinates control within regions, also failed to converge after 200 episodes in a large-scale road network. However, RegionLight performed well on the Jinan road network at 200 episodes, with a very short average travel time from the start. Colight showed rapid initial convergence, indicating an effective use of traffic network information to quickly find a reasonable TSC policy, although it could not learn the optimal policy. Compared to the other five methods, HAMH-PPO excels in both convergence speed and final learning results. Thus, we conclude that the HAMH-PPO model can learn better policies and achieve excellent convergence speed.

In addition to performance, we also compared the training time and memory usage of different methods on the synthetic dataset. As shown in Table 4, HAMH-PPO achieves competitive efficiency, with a training time of only 1.5 hours and RAM usage of 2.0 GB, significantly lower than

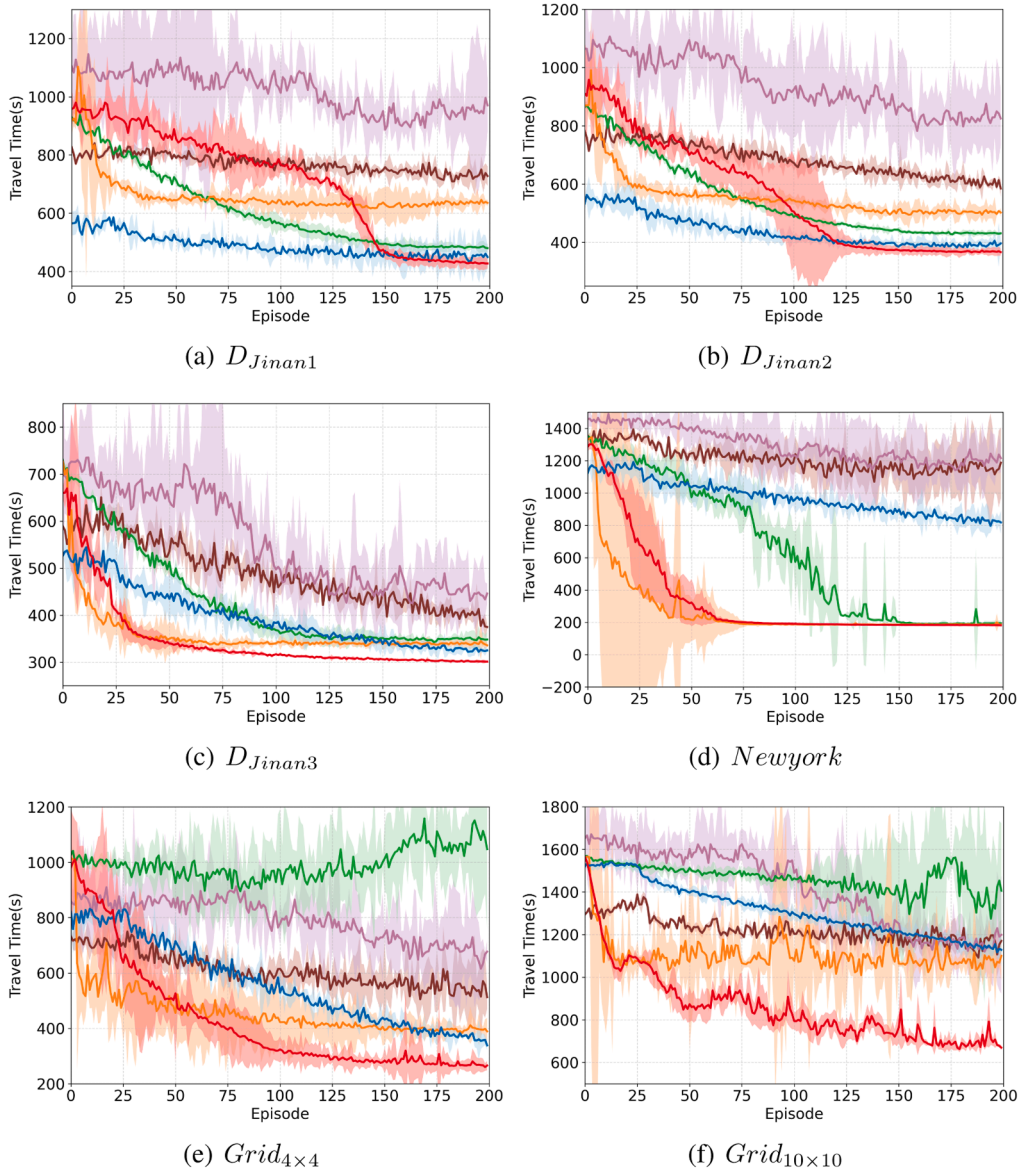


Fig. 6. Performance comparison of RL methods (RegionLight ‘—’, PNC-HDQN ‘—’, MA2C ‘—’, Colight ‘—’, MPLight ‘—’ and HAMH-PPO ‘—’) in six datasets.

Table 4
Training time and memory usage on Jinan3 data.

Methods	PNC-HDQN	MA2C	MPLight	Colight	RegionLight	HAMH-PPO
Training Time	4.0	1.6	1.2	<u>1.4</u>	2.9	1.5
RAM	8.0	2.2	<u>2.0</u>	1.5	3.0	<u>2.0</u>

methods such as PNC-HDQN (4.0 hours, 8.0 GB). This demonstrates that HAMH-PPO not only outperforms existing methods in terms of traffic efficiency but also achieves competitive computational efficiency, making it a practical choice for real-world applications.

5.6. Personalized policy verification of HAMH-PPO (RQ2)

In this section, we validate the effectiveness of the proposed hyper-action mechanism by visualizing the diversity of the HAMH-PPO policy network. To demonstrate this, we statistically analyze the scatter plots of the action distributions for both HAMH-PPO and PPO after convergence. The eight-dimensional action distribution data is reduced to two dimensions using PCA and shown in Fig. 7. The figure shows the action distributions at three intersections (IDs 1, 4, and 9) in D_{Jinan1} . The ac-

tion distributions in HAMH-PPO were distinctly dispersed, while those in PPO were relatively concentrated with significant overlap between different intersections. This indicates that HAMH-PPO effectively learns a diverse policy network, significantly improving traffic efficiency in the road network.

We further tested the impact of hyper-actions on intersection policies in the trained HAMH-PPO model. We measured traffic flows (vehicles/300s) at two intersections, namely 1 and 11. As shown in Fig. 8(a), intersection 1 has a higher traffic volume than intersection 11. During the time intervals of 1200 to 1500 and 3000 to 3300, the traffic flows at both intersections are very similar. We set the hyper-action output dimension to 2 and computed the probability values for the hyper-actions taken by the agent at each step. In the simulator, actions are sampled every ten seconds, so the horizontal axis ranges from 0 to 360. As shown

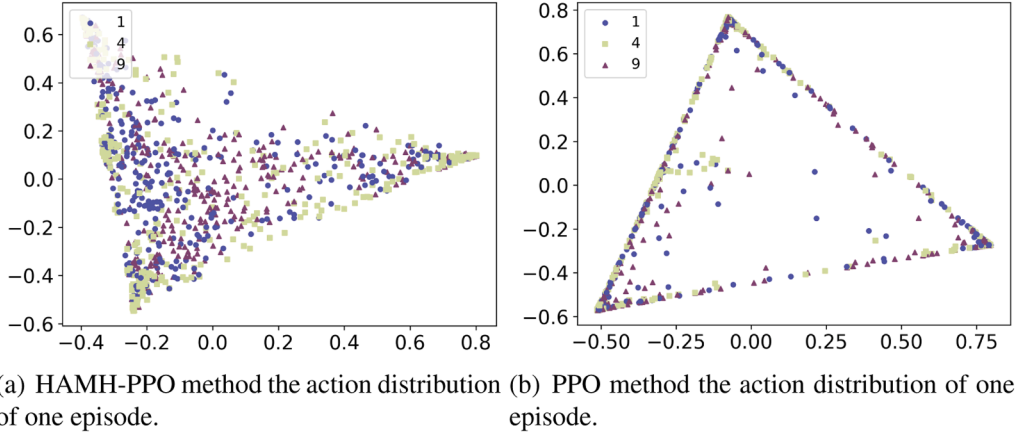


Fig. 7. Statistical results of the action distribution for three intersections (IDs 1, 4, and 9) in the D_{Jinan1} dataset for HAMH-PPO and PPO. The statistical results of one episode (360 steps) after the convergence of the two methods.

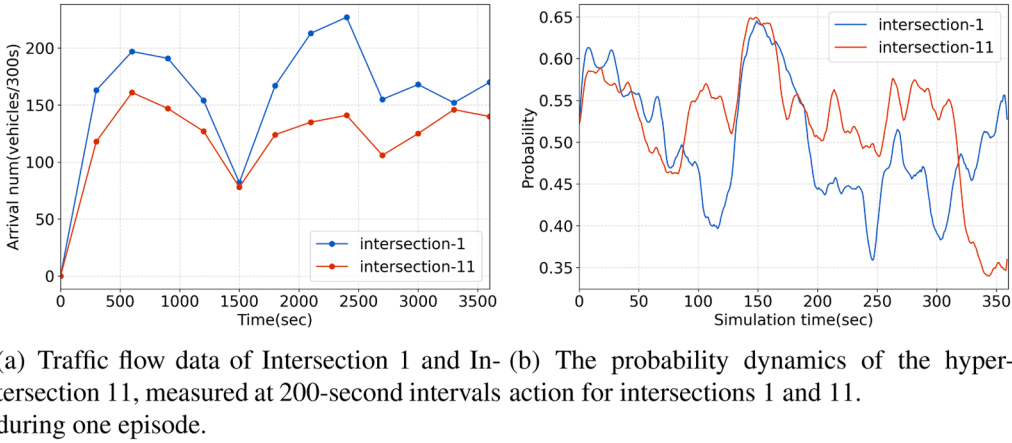


Fig. 8. Effectiveness of hyper-action in HAMH-PPO (dataset: D_{Jinan3}).

in Fig. 8(b), we plotted only the first dimension of the hyper-action values. The probability values at both intersections are extremely similar at the 150th step (i.e., 1500s), but change in opposite directions after the 330th step (i.e., 3300s), consistent with the traffic flow distribution at the two intersections. This shows that the hyper-action mechanism can dynamically adjust the importance of value functions across different environments, thereby accounting for preferences among intersections.

5.7. Ablation study (RQ3)

5.7.1. The effectiveness of the components

To verify the effectiveness of each component in HAMH-PPO, this section conducts an ablation study. The goal is to determine whether the algorithm's overall performance changes significantly when key modules are removed. The experiments were carried out on two datasets: D_{Jinan1} , with a high traffic density, and $Grid_{4 \times 4}$, with a high variance. HAMH-PPO features a multi-head value function library that provides differentiated value function estimations for the traffic network. A hyper-network dynamically generates adaptive weights for these value functions, enabling customized control strategies for each intersection. Hyper-actions capture intersectional preferences, while entropy ensures uniform updates to the value function, promoting effective learning. The HAMH-PPO algorithm comprises four main modules: basic PPO, MH-Critic, HA-Actor, and weight entropy. PPO serves as the base algorithm. PPO + MH-Critic adds multi-head value functions, train-

ing N value functions without sharing the critic network's last-layer MLP. PPO + hyper represents HAMH-PPO without weight entropy, while PPO + hyper + entropy is the complete HAMH-PPO algorithm. The results show significant performance improvements in both road networks when multi-head value functions are added to PPO. In the D_{Jinan1} network, adding the hyper-action network boosts performance by 4.5%, while entropy has a minor impact. Entropy prevents weights from stabilizing too quickly, slightly improving performance. The experiment confirms that each module contributes effectively to the overall performance.

5.7.2. The effectiveness of the dimensions in hyper-action

To further verify the influence of the hyper-action dimension, we set the value functions in different dimensions to compare the training results. Given that the complexity of collaborative scenarios is directly influenced by traffic density, we selected D_{Jinan1} , D_{Jinan2} , and $Grid_{4 \times 4}$, which feature heavy traffic flows. In Fig. 10(a), we calculate the travel time when different value function counts converged for three traffic flow distributions in two traffic environments. The line graph represents the average of three training results, and the orange line in the box denotes the median. The experimental results indicate that the dimension of hyper-action has the best effect around 32. This can be explained by a trade-off between representational capacity and estimation stability. When k is too small (e.g., 1, 2 or 16), the set of value functions lacks sufficient capacity to capture the diverse characteristics of different

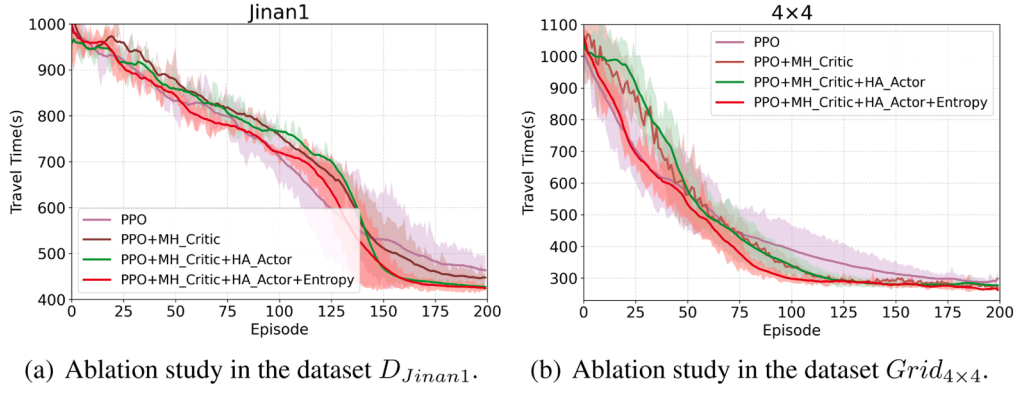


Fig. 9. Learning dynamics of PPO + hyper + Entropy, PPO + hyper and PPO in two traffic flow distributions.

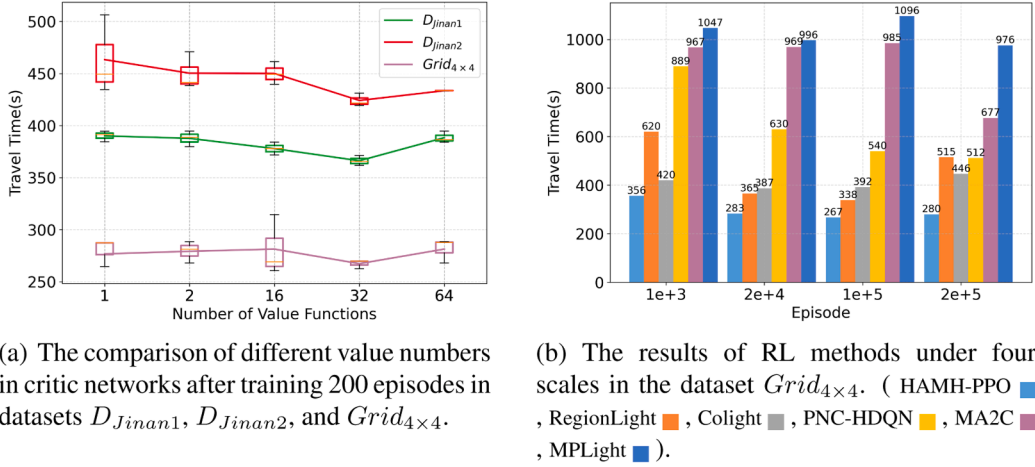


Fig. 10. Experimental comparisons of algorithms in the dimensions of hyper-action and neural network parameter scale.

intersections, leading to underfitting and degraded performance. When k is too large (e.g., 64), the increased number of value functions introduces redundancy and makes the weight estimation more prone to overfitting, especially given the limited training samples per intersection. The choice of $k = 32$ strikes a balance between these two factors, providing enough representational flexibility while maintaining stable training. This trade-off is consistent with similar observations in multi-head attention mechanisms, where a moderate number of heads typically yields optimal performance.

5.7.3. Comparison of neural network parameter scale

Our algorithm combines the advantages of both shared- and non-shared parameter algorithms. Sharing parameters can improve the efficiency of model training, but it does not account for the preferences of each intersection. Non-parameter sharing trains a separate set of parameters for each agent, and the parameter scale becomes very large when there are many intersections in the environment. Our algorithm retains the advantages of parameter sharing while mitigating its disadvantages, achieving good results even with a smaller-scale neural network. We compared the performance of five reinforcement learning-based methods on four different parameter scales. As shown in Fig. 10(b), our algorithm performs the best on all four network scales. Especially when the parameter scale is 1000, HAMH-PPO achieves an effect that other algorithms cannot achieve. Additionally, when the parameter scale is small, all algorithms fail to identify the best policy because the neural network cannot fully represent the intersection's features. As the number of neural network parameters increases, the performance of shared-parameter algorithms (HAMH-PPO, Colight, MPLight) initially improves and then stabilizes. The performance of HAMH-PPO and RegionLight de-

clines when the number of parameters is very large, which we consider an overfitting problem. Increasing the weight entropy coefficient can help alleviate neural network overfitting.

6. Conclusion

The Hyper-Action Multi-Head Proximal Policy Optimization (HAMH-PPO) algorithm proposed in this paper offers an effective solution for TSC in large-scale intersection scenarios. By introducing the concepts of hyper-action and multi-value estimation, HAMH-PPO achieves personalized policy learning for intersections with non-independent and identically distributed (non-iid) observational distributions, while maintaining efficient parameter sharing. The experimental results demonstrate that HAMH-PPO improved traffic efficiency while ensuring efficient training. Leveraging hyper-action from the HA-Actor, HAMH-PPO can learn preferential policies without increasing the number of network parameters, thus better accommodating the specific needs of different intersections. Furthermore, the critic network, integrated with multi-value estimation, provides a more accurate assessment of the average travel time at each intersection, thereby improving policy performance. By efficiently sharing parameters and learning personalized policies, the overall parameter count is significantly reduced, thereby lowering deployment costs and enhancing the algorithm's feasibility and efficiency in practical applications.

Future research will focus on enriching the decision-making context of the hyper-action. A primary direction is the deep integration of real-time traffic-demand prediction models, enabling the hyper-action to dynamically adjust the weights of value-function combinations based on

short-term traffic-flow forecasts. Additionally, lightweight inter-agent communication protocols will be investigated to facilitate the exchange of predictive information and local states among intersections. The hyper-action would then integrate ‘local observations,’ ‘predictive information,’ and ‘neighborhood intents’ to form a multi-dimensional decision-making foundation. The ultimate goal is to shift from reactive responses to proactive cooperation, enabling the entire network to collectively absorb traffic fluctuations and maximize overall efficiency. Fig. 9.

CRedit authorship contribution statement

Kailing Zhou: Writing – original draft, Validation, Resources, Methodology, Investigation, Formal analysis, Data curation; **Chengwei Zhang:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization; **Furui Zhan:** Writing – review & editing, Supervision, Formal analysis, Data curation; **Wanting Liu:** Methodology; **Yihong Li:** Visualization, Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant 62376048 and the Shandong Provincial Natural Science Foundation, China under Grant ZR2022LZH002.

References

- Abdoos, M., & Bazzan, A. L. C. (2021). Hierarchical traffic signal optimization using reinforcement learning and traffic prediction with long-short term memory. *Expert Systems with Applications*, 171, 114580.
- Chen, C., Wei, H., Xu, N., Zheng, G., Yang, M., Xiong, Y., Xu, K., & Li, Z. (2020). Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3414–3421). (vol. 34).
- Chen, W., Yang, S., Li, W., Hu, Y., Liu, X., & Gao, Y. (2024). Learning multi-intersection traffic signal control via coevolutionary multi-agent reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, (pp. 1–17).
- Christianos, F., Papoudakis, G., Rahman, M. A., & Albrecht, S. V. (2021). Scaling multi-agent reinforcement learning with selective parameter sharing. In *International conference on machine learning* (pp. 1989–1998). PMLR.
- Chu, T., Wang, J., Codeca, L., & Li, Z. (2019). Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 21(3), 1086–1095.
- Devailly, F.-X., Larocque, D., & Charlin, L. (2021). Ig-rl: Inductive graph reinforcement learning for massive-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 23(7), 7496–7507.
- Fang, Z., Zhang, F., Wang, T., Lian, X., & Chen, M. (2022). Monitorlight: Reinforcement learning-based traffic signal control using mixed pressure monitoring. In *Proceedings of the 31st ACM international conference on information & knowledge management* (pp. 478–487).
- Ge, H., Gao, D., Sun, L., Hou, Y., Yu, C., Wang, Y., & Tan, G. (2021). Multi-agent transfer reinforcement learning with multi-view encoder for adaptive traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 23(8), 12572–12587.
- Gu, H., Wang, S., Ma, X., Jia, D., Mao, G., Lim, E. G., & Wong, C. P. R. (2024). Large-scale traffic signal control using constrained network partition and adaptive deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, 25(7), 7619–7632.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30, 1024–1034.
- Huang, H., Hu, Z., Lu, Z., & Wen, X. (2021). Network-scale traffic signal control via multi-agent reinforcement learning with deep spatiotemporal attentive network. *IEEE Transactions on Cybernetics*, 53(1), 262–274.
- Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S., Stich, S., & Suresh, A. T. (2020). SCALFOLD: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th international conference on machine learning* (pp. 5132–5143). (vol. 119).
- Koonce, P., Rodegierds, L., Lee, K., Quayle, S., Beard, S., Braud, C., Bonneson, J. A., Tarnoff, P., & Ii, T. (2008). Traffic signal timing manual.
- Liu, S., Zhou, Y., Song, J., Zheng, T., Chen, K., Zhu, T., Feng, Z., & Song, M. (2023a). Contrastive identity-aware learning for multi-agent value decomposition. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 11595–11603). (vol. 37).
- Liu, Y., Luo, G., Yuan, Q., Li, J., Jin, L., Chen, B., & Pan, R. (2023b). Gplight: Grouped multi-agent reinforcement learning for large-scale traffic signal control. In *Proceedings of the thirty-second international joint conference on artificial intelligence* (pp. 199–207).
- Ma, J., & Wu, F. (2020). Feudal multi-agent deep reinforcement learning for traffic signal control. In *Proceedings of the 19th international conference on autonomous agents and multiagent systems (AAMAS)* (pp. 816–824).
- Mukhtar, H., Afzal, A., Alahmari, S., & Yonbawi, S. (2023). Ccgn: Centralized collaborative graphical transformer multi-agent reinforcement learning for multi-intersection signal free-corridor. *Neural Networks*, 166, 396–409.
- Qi, L., Sun, Y., & Luan, W. (2024). Large-scale traffic signal control based on multi-agent q-learning and pressure. *IEEE Access*, 12, 1092–1101.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. In *4th international conference on learning representations*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.
- Shen, J., Hu, J., Zhao, Q., & Rao, W. (2023). Heterogeneous traffic intersections control design based on reinforcement learning. *IET Intelligent Transport Systems*, (pp. 1–17).
- Varaiya, P. (2013). The max-pressure controller for arbitrary networks of signalized intersections. In *Complex Networks and Dynamic Systems, Advances in Dynamic Network Modeling in Complex Transportation Systems*, pp. 27–66.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y. et al. (2017). Graph attention networks. *Stat*, 1050(20), 10–48550.
- Wang, Y., Xu, T., Niu, X., Tan, C., Chen, E., & Xiong, H. (2020). Smtar: A spatio-temporal multi-agent reinforcement learning approach for cooperative traffic light control. *IEEE Transactions on Mobile Computing*, 21(6), 2228–2242.
- Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019a). Presslight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1290–1298).
- Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., & Li, Z. (2019b). Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1913–1922).
- Wei, H., Zheng, G., Yao, H., & Li, Z. (2018). Intellilight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2496–2505).
- Wu, Q., Zhang, L., Shen, J., Lü, L., Du, B., & Wu, J. (2021). Efficient pressure: Improving efficiency for signalized intersections. *CoRR*, abs/2112.02336.
- Xu, B., Wang, Y., Wang, Z., Jia, H., & Lu, Z. (2021). Hierarchically and cooperatively learning traffic signal control. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 669–677). (vol. 35).
- Yan, L., Zhu, L., Song, K., Yuan, Z., Yan, Y., Tang, Y., & Peng, C. (2023). Graph cooperation deep reinforcement learning for ecological urban traffic signal control. *Applied Intelligence*, 53(6), 6248–6265.
- Yang, S., Yang, B., Kang, Z., & Deng, L. (2021). Ihg-ma: Inductive heterogeneous graph multi-agent reinforcement learning for multi-intersection traffic signal control. *Neural Networks*, 139, 265–277.
- Yang, S., Yang, B., Zeng, Z., & Kang, Z. (2023). Causal inference multi-agent reinforcement learning for traffic signal control. *Information Fusion*, 94, 243–256.
- Zang, Y., He, J., Li, K., Fu, H., Fu, Q., Xing, J., & Cheng, J. (2024). Automatic grouping for efficient cooperative multi-agent reinforcement learning. *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 36.
- Zhang, C., Jin, S., Xue, W., Xie, X., Chen, S., & Chen, R. (2021). Independent reinforcement learning for weakly cooperative multiagent traffic control problem. *IEEE Transactions on Vehicular Technology*, 70(8), 7426–7436.
- Zhang, C., Tian, Y., Zhang, Z., Xue, W., Xie, X., Yang, T., Ge, X., & Chen, R. (2022a). Neighborhood cooperative multiagent reinforcement learning for adaptive traffic signal control in epidemic regions. *IEEE Transactions on Intelligent Transportation Systems*, 23(12), 25157–25168.
- Zhang, F., Yang, Q., & An, D. (2022b). A leader-following paradigm based deep reinforcement learning method for multi-agent cooperation games. In *Neural networks* (pp. 1–12). (vol. 156).
- Zheng, G., Xiong, Y., Zang, X., Feng, J., Wei, H., Zhang, H., Li, Y., Xu, K., & Li, Z. (2019). Learning phase competition for traffic signal control. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1963–1972).